

Financial Ratio Analysis Engine

An Automated Ratio Computation & Financial Health Scoring System for Nifty 100 Companies

Database	SQLite — nifty100.db
Companies Covered	Nifty 100 Index Constituents
Records Processed	1,209 rows × 41 columns
Core Tools	Python, pandas, SQLite3

1. Objective

The objective of this project is to design and implement an end-to-end data pipeline that consolidates raw financial statement data for Nifty 100 companies and automatically derives a standardized set of financial ratios. The system is intended to support quick, consistent, and scalable evaluation of a company's profitability, efficiency, and overall financial health across multiple years, without relying on manual spreadsheet calculations.

Specifically, the project aims to:

- Store multi-year financial statement data (Profit & Loss, Balance Sheet, Cash Flow, Market Cap, and Analyst data) in a structured relational database.
- Merge these disparate data sources into a single, analysis-ready dataset keyed by company and year.
- Build a reusable **Ratio Engine** that computes key profitability and efficiency ratios directly from raw financials.
- Derive a composite **Financial Health Score** that summarizes a company's standing in a single metric.
- Persist the computed ratios back into the database for downstream querying and reporting.

2. Database Design

All data is stored in a single SQLite database file, **nifty100.db**, chosen for its lightweight, file-based, and serverless nature — well suited for a project of this analytical scope. The design follows a star-like structure, where individual statement tables are merged on a common key (**company_id** and **year**, tied together via an **id_pl** reference) into one wide, denormalized table optimized for ratio computation.

2.1 Source Tables

Table	Contents	Rows
Profit & Loss	sales, expenses, operating profit, OPM %, other income, interest, depreciation, profit before tax, tax %, net income	1,079
Balance Sheet	equity capital, reserves, borrowings, other liabilities, total liabilities, fixed assets, CWIP, investments, other assets	1,079
Cash Flow	operating activity, investing activity, financing activity, net cash flow	1,065
Market Cap	market cap (crore), enterprise value, PE / PB / EV-EBITDA ratios, dividend yield %	1,065
Analysis	compounded sales growth, compounded profit growth, stock price CAGR, ROE	729

2.2 Merge Pipeline

The four source tables are progressively left-merged into a single consolidated dataset. Each merge step widens the table with additional financial dimensions while preserving one row per company-year:

Step	Merge	Resulting Shape
1	Profit + Balance	(1,079, 26)
2	+ Cash Flow	(1,065, 31)

3	+ Market Cap	(1,065, 37)
4	+ Analysis	(1,209, 41)

The final merged table contains **1,209 rows and 41 columns**, giving complete year-wise financial coverage per company and forming the direct input to the Ratio Engine.

3. Ratio Engine

The Ratio Engine is a Python module (built on pandas) that reads the merged financial dataset, computes a standardized set of financial ratios row-by-row (i.e. per company per year), and writes the results into a dedicated **financial_ratios** table in the database. It is designed to run independently of the merge step, so ratios can be recalculated at any time without re-processing the raw statements.

Processing flow:

- Load the merged financial dataset from the database.
- Apply vectorized pandas calculations to derive each ratio.
- Handle missing / non-computable values gracefully (e.g. NaN where interest expense is zero, in early years for ABB).
- Persist the resulting ratio table back to SQLite via **financial_ratios**.
- Print a verification preview and row count as a sanity check after saving.

In the executed run, the engine successfully saved all **1,209 rows** to the financial_ratios table, confirming full coverage of the merged dataset.

4. Financial Ratios

The financial_ratios table stores nine columns per company-year: identifying fields (company_id, year) and seven computed metrics spanning profitability, leverage, efficiency, and cash generation.

Ratio	Description
Net Profit Margin %	Net profit as a share of total sales — overall profitability.
Return on Equity (ROE) %	Net profit relative to shareholders' equity — return generated for owners.
ROCE %	Operating profit relative to capital employed — efficiency of all capital used.
ROA %	Net profit relative to total assets — how efficiently assets generate profit.
Interest Coverage	Operating profit relative to interest expense — ability to service debt.
Asset Turnover	Sales generated per unit of total assets — asset utilization efficiency.
Free Cash Flow (Cr)	Cash generated after operating and investing activities.

Sample Output — Company ABB

Company	Year	Net Profit Margin %	ROE %	ROCE %	ROA %	Health Score
ABB	Dec 2012	8.77	22.41	31.22	15.99	3
ABB	Mar 2014	8.70	25.13	33.88	17.38	3
ABB	Mar 2015	10.00	24.44	33.30	16.67	3
ABB	Mar 2016	9.76	21.34	30.54	15.78	4
ABB	Mar 2017	9.54	19.97	28.70	13.41	4

Extract from the financial_ratios table, generated directly from the SQLite database.

5. Return on Capital Employed (ROCE)

ROCE measures how efficiently a company uses *all* the capital available to it — both equity and borrowed funds — to generate operating profit. Because it accounts for debt as well as equity, it is widely regarded as a more complete profitability measure than ROE alone, particularly for capital-intensive businesses.

$$ROCE (\%) = \text{Operating Profit} / \text{Capital Employed} \times 100$$

where Capital Employed = Equity Capital + Reserves + Borrowings. In the sample output for ABB, ROCE ranged from roughly **28.7% to 33.9%** between 2012 and 2017, indicating strong and fairly stable capital efficiency over that period.

6. Return on Assets (ROA)

ROA measures how efficiently a company converts its total asset base into net profit. It is a useful complement to ROE and ROCE, as it is unaffected by the company's specific financing mix and instead reflects pure operational asset efficiency.

$$ROA (\%) = \text{Net Profit} / \text{Total Assets} \times 100$$

For ABB, ROA moved from **15.99% in 2012 to 17.38% in 2015**, before easing to **13.41% by 2017**, suggesting a period of asset base expansion that modestly outpaced profit growth in the later years.

7. Financial Health Score

To make cross-year and cross-company comparison easier, the engine condenses the full ratio profile into a single composite **Financial Health Score**. The score aggregates signals from profitability (net profit margin, ROE), efficiency (ROCE, ROA, asset turnover), and solvency (interest coverage) into a compact rating, allowing a company's overall trend to be read at a glance without inspecting every individual ratio.

In the ABB sample, the score improved from **3 (2012–2015)** to **4 (2016–2017)**, consistent with the ratio movements observed above and reflecting a strengthening overall financial position over the period.

8. Results

- Successfully consolidated five raw financial data sources into one unified dataset of **1,209 rows × 41 columns**.
- Ratio Engine executed without errors, computing all seven core financial ratios for every company-year record.
- All **1,209 rows** were persisted to the `financial_ratios` table and verified via direct SQL query.
- Sample verification (company ABB) confirmed consistent, sensible ratio values across an 5+ year span, with expected NaN values only where source data (e.g. interest expense) was unavailable in early years.
- The pipeline runs end-to-end from raw statement tables to a query-ready ratios table with a single script execution.

9. Conclusion

This project demonstrates a complete, reproducible workflow for transforming raw, multi-source financial statement data into decision-ready financial ratios and a composite health score, using a lightweight SQLite database and a pandas-driven ratio engine. The resulting `financial_ratios` table provides a solid foundation for further analysis — such as peer comparison across Nifty 100 companies, trend analysis over time, or as an input feature set for predictive financial health models.

Future extensions could include automated sector-wise benchmarking, visualization dashboards built directly on the `financial_ratios` table, and refinement of the Financial Health Score using weighted or machine-learning-based scoring rather than a fixed rule set.